SDP

FanHub Unplugged

Fix It, Ship It, Vibe It

60 minutes of agent-directed development — you hold the vision, the agent implements

Workshop · 60 minutes

SECTION 1

The World & The One Rule

What you're working with — and how you'll work

📦

Meet FanHub

Deliberately broken — that's the point

🎬

The One Rule

Director, not Implementer

🌍 The World & The One Rule

What is FanHub?

A deliberately broken, underdocumented, multi-language fan site starter — not a toy

Four Complete Implementations

Node.js
Express + React + PostgreSQL

.NET
ASP.NET Core + Blazor + PostgreSQL

Java
Spring Boot + React + PostgreSQL

Go
Gin + React + PostgreSQL

Intentional Problems — Identical Across All Tracks

No documentation — agents can't orient without context

Known bugs — duplicate characters, broken filters, inconsistent APIs

Generic UI — unstyled, unthemed, crying out for personality

Incomplete features — scaffolding everywhere, implementations nowhere

"Four languages, same bugs. That's actually kind of beautiful in a terrible way." — Marcus

The One Rule

The single behavioral shift that governs everything that follows

You are the Director.

The agent is the Implementer.

"You fully give in to the vibes, embrace exponentials, and forget that the code even exists."

— Andrej Karpathy, Collins Dictionary Word of the Year 2025

❌ If you're doing this...

- Reading source files line by line
- Manually editing the agent's code
- Debugging implementation details yourself

You've left director mode.

✅ Your actual job is...

- Hold the vision — know what done looks like
- Communicate clearly — describe, don't implement
- Redirect when needed — steer with words

The agent is faster at code than you are.

SECTION 2

Onramp: Pick, Init, Run

Get every participant to the same starting line (0:00 – 0:20)

🎯

Pick Your Stack

Track + surface in 3 minutes

🧭

/init and Orient

Let the agent read the codebase

▶️

Build and Run

App running in the browser

Onramp

Pick Your Stack (0:00 – 0:03)

Two decisions — track and surface. Both happen right now.

Choose Your Language Track

- **Node.js** — Express + React + PostgreSQL
- **.NET** — ASP.NET Core + Blazor + PostgreSQL
- **Java** — Spring Boot + React + PostgreSQL
- **Go** — Gin + React + PostgreSQL

Bugs are identical — pick the language you know best.

Choose Your Surface

VS Code — Copilot Chat (Agent Mode)

Open Copilot Chat → switch mode to `Agent` → full tool access: reads files, writes files, runs terminals

Copilot CLI — Terminal Native

`copilot` — `Shift+Tab`

Both surfaces: same model, same tools, same interaction model. Pick what feels natural.

🧭 Onramp

2 of 4

/init and Orient (0:03 – 0:10)

Before the agent does anything useful, it needs to understand the codebase — and your track

Run /init with Your Track

```
# Node.js:
/init I'm working in the node/ track

# .NET:
/init I'm working in the dotnet/ track

# Java:
/init I'm working in the java/ track

# Go:
/init I'm working in the go/ track
```

Then Ask for Priorities

"What are the three most impactful things I could fix or build in the next 45 minutes?"

Why This Matters

✅ With track hint: agent scopes to your working tree

❌ Without it: reads all four implementations and produces broad, unfocused orientation

/init is not optional. It's the foundation of every useful agent session.

Onramp

3 of 4

Build and Run (0:10 – 0:20)

Get the app running — use the agent to fix setup errors

Simple Ask

> *"Walk me through getting this running from the beginning."*

Common Setup Gotchas

Node: npm run install:all at root

.NET: docker-compose up -d db first

Java: Frontend/backend on different ports

Go: docker-compose up --build

This Is Not a Detour

✅ Using the agent to fix setup errors is the **first real demonstration of the loop**

✅ The agent reads the error, identifies the cause, and fixes it — without you touching a file

Explore the Bugs Briefly

- Two Jesse Pinkmans in the character list
- A season filter that doesn't filter
- Generic UI crying out for personality

Not to fix them yet — just to give the agent context

✅ Onramp

🙋 Is everyone running?

Facilitator: circulate the room now. This slide stays up until everyone answers yes.

- ☐ **Codespace open at github.com/MSBart2/FanHub**
  Dev container for your track is running
- ☐ **/init run and scoped to your track**
  `/init I'm working in the node/ track`
  Replace node/ with dotnet/, java/, or go/ as needed
- ☐ **App is running in the browser**
  Characters, episodes, and at least two Jesse Pinkmans. You're ready.

Not running yet? Use the agent: `"Walk me through getting this running from the beginning."`

SECTION 3

Sprint: Choose Your Adventure

28 minutes of agent-directed flow (0:20 – 0:48)

🐛

Bug Blitz

Fix clusters from BUGS.md

🎨

Full Rebrand

Pick a show, transform the UI

✨

Build Something New

End-to-end feature from scratch

🎉

Delight Mode

Trivia games, matchups, personality

Pick a Path (0:20 – 0:25)

Five minutes. Give the agent a clear mandate that lets it run for 20+ minutes with minimal interruption.

Bug Blitz

Batch a cluster from BUGS.md — not one at a time

```
"Fix the three highest-severity bugs in BUGS.md. For each one, explain the root cause before writing the fix."
```

Full Rebrand

Pick any show — colors, copy, cards, tagline — everything in one shot

```
"Rebrand this as a [YOUR SHOW] fan site. Matching palette, flavor text, on-brand cards, iconic tagline – one pass."
```

Build Something New

End-to-end spec: route, API, and frontend

```
"Add full-text search across characters and episodes. Search box in nav, dropdown results as you type, API ?q= param. Build both layers end-to-end."
```

Delight Mode

Pick what sounds fun — then

```
"Build a 'Who Said It?' trivia game at /trivia. Random quote, 4 character options, score tracking."
```

Sprint

2 of 2

The Build Sprint (0:25 – 0:48)

23 minutes of agent output. Your job is to direct — not implement.

✅ Director Mode

- Describe the goal clearly upfront — give the agent a complete spec
- Let it run for 5+ minutes without interrupting
- When it pauses: answer with specifics, not vague redirects
- When it goes wrong: describe the problem, let the agent fix itself

❌ Implementer Mode (avoid)

- Reading generated code line by line before it finishes
- Manually editing files the agent is mid-way through
- Pre-analyzing errors before pasting them in

The Error-Paste Pattern

Get an error? Paste it into chat with zero explanation. The agent has the context — it will almost always fix it.

"If you're stuck more than 2 minutes, it's a prompt problem — not a Copilot problem." Restate the goal, add constraints, show the agent the error.

SECTION 4

Ship & Zoom Out

Polish, demo, and the honest reckoning (0:48 – 1:00)

✨

Polish

README and final review

🎬

Show and Tell

Director's Commentary

⚖️

Zoom Out

What the data actually says

✨ Ship & Zoom Out

1 of 3

Polish & Show and Tell (0:48 – 1:00)

Finish what you started, document what you built, then show it off

README Update

"Write a README section documenting what we built — what the feature/fix is, how to use it, any technical details."

Final Review

"Review what we built. What's the most brittle or incomplete part? Fix it."

🎬 Show and Tell (0:57 – 1:00)

Everyone demos one thing. Most creative use of the agent wins.

- 🤯 Show the prompt that generated the most code
- 🦠 A bug where the agent found the root cause faster than you would have
- 🎨 Before/after that makes the old version look embarrassing
- 🎉 The most unhinged thing you shipped

🚢 Ship & Zoom Out

2 of 3

The Honest Tradeoffs

Vibe coding is genuinely useful for prototyping and sprints — it is not a free lunch in production

📊 The Research

METR 2025: Experienced devs were 19% slower with AI on complex OSS tasks — despite predicting 24% faster

CodeRabbit 2025: AI co-authored code had 1.7x more major issues, 2.74x more security vulnerabilities

Simon Willison: "Vibe coding your way to a production codebase is clearly risky."

💬 The Voices

"The METR study is doing a lot of work here. 'Experienced developers on complex OSS tasks' ≠ 'me telling an agent to finish a trivia game in 20 minutes.'"

— Rafael

"I accept the tradeoffs for a sprint. I would not accept them for the auth layer in a production system."

— David

"I verified every fix against BUGS.md. 3 out of 4 were correct and one needed a follow-up. That's not magic but it's not nothing."

— Elena

FanHub is the right context for vibe coding. The goal is to build the skill of directing agents — including knowing when to slow down.

What would you actually use this for tomorrow?

The habits transfer immediately. The question is where you point them.

Use /init before every agent session

On any unfamiliar codebase [illegible] first

Batch your requests

Thematic clusters beat single-item back-and-forth

Write complete prompts

Goal + constraints + [illegible]

Direct, don't implement

If you're editing code [illegible] could be doing it

✨ **You did this with minimal Copilot configuration.**

/init gave the agent basic orientation — no custom instructions, no skills, no architecture files. **Imagine what a properly configured workspace looks like.**

→ Copilot Primitives — what configuration actually unlocks

You shipped it.

An hour ago this was someone else's codebase. Now it's yours.

🎤

Show and Tell

Demo one thing — most creative use wins bragging rights

🧠

The Skill You Built

Directing an agent — that transfers to every project from here

🚀

Keep Going

github.com/MSBart2/FanHub — the repo is yours

The Only Metric That Matters

"Did you ship something you're excited to demo? If yes, you won. If no, you still learned how to direct an agent through 23 minutes of uninterrupted work — and that's the transferable skill."